# Rahul Koju

DevOps Engineer

9840456322 | Bhaktapur, Nepal | contact@rahulkoju.com.np | Portfolio | GitHub | LinkedIn

## PROFESSIONAL SUMMARY

Hands-on experience building containerised applications, CI/CD pipelines, and cloud infrastructure on AWS. Operate production-grade Kubernetes clusters with deployments automated via GitOps. IaC via Terraform & Ansible to provision resources on AWS & environment respectively. Run Prometheus, Grafana, and Loki for observability and harden network security with Ingress, Kubernetes NetworkPolicies & IAM. Actively seeking a DevOps internship to contribute my skills and keep growing in infrastructure, automation, and cloud operations.

## SKILLS

**CI/CD & GitOps:** GitHub Actions, Jenkins, ArgoCD
**Container Orchestration:** Docker, Docker Compose, Kubernetes via RKE & EKS, Helm
**Cloud & IaC:** AWS (EC2, VPC, IAM, S3, Security Groups), Terraform, Ansible
**Observability:** Prometheus, Grafana, Loki, Alertmanager
**Networking & Security:** Firewall Configuration, TLS/SSL, Subnetting
**Backend:** NodeJS/Bun/Deno
**Database:** Postgres, MariaDB/mySQL, MongoDB
**Frontend:** ReactJS, NextJS, Vite
**Languages:** Golang, TypeScript, Bash, Python

## WORK EXPERIENCE

### Software Developer

**Feb 2025 - May 2026**

*Lipi Point Pvt. Ltd., Bhaktapur, Nepal*

- Cut container image sizes by 92% (3–4 GB to 200–350 MB) and reduced EC2 cold-start time by ~15s by authoring multi-stage Dockerfiles with Turborepo pruning across a 3-service monorepo, enabling faster, lighter production deployments.
- Eliminated environment-drift incidents across dev and production by configuring Docker Compose with isolated bridge networking, restart policies, and environment-variable injection across 3 services, achieving consistent, repeatable deployments.
- Reduced production misconfiguration incidents by ~80% by redesigning REST API config to environment-variable separation with structured error handling, replacing a single-config setup that caused repeated deployment failures.
- Cut mean time to resolve (MTTR) production incidents by ~70% by standardizing Docker health checks and structured JSON logging across all 3 Next.js services, enabling faster log analysis and reducing average debug-to-fix cycles from hours to minutes.
- Automated image optimisation, gallery generation, and asset pipeline tasks using Python and Bash scripts, eliminating ~2 hours of manual work per release and reducing per-deploy asset processing time by ~40%.
- Authored deployment runbooks, onboarding SOPs, infrastructure setup guides, and system architecture docs across 2 production projects, reducing new team member ramp-up time by ~50% and establishing a reliable incident response reference used across the team.

## PROJECTS

### Commit – Three-Tier Web Application Github | Live

**Jan 2026 - Present**

Personal productivity app (tasks, habits, notes, focus timer) built with React, Go, and Postgres, deployed on Kubernetes with GitOps, full observability, and automated CI/CD on AWS.

- Provisioned the full AWS environment (VPC, networking, security groups, IAM resources) with Terraform in ~50 seconds, cutting infrastructure setup from a multi-hour manual process to a single repeatable script.

- Automated Linux node configuration (swap, kernel modules, sysctl tuning) with Ansible in ~65s and bootstrapped a 2-node RKE Kubernetes cluster with Canal CNI and NGINX Ingress in ~2m 25s, cutting total cluster setup from a multi-hour manual process to under 10 minutes from a blank AWS environment to a fully operational, monitored production stack.
- Eliminated configuration drift across all environments by authoring Kubernetes manifests for 3 application tiers with ConfigMaps, Secrets, and a Postgres StatefulSet backed by persistent storage, replacing ad-hoc kubectl changes with fully declarative, version-controlled configuration.
- Built a GitHub Actions CI/CD pipeline that builds Docker images tagged with commit SHA and auto-updates deployment manifests, reducing release effort to zero with consistent sub-3-minute deployments.
- Implemented GitOps delivery with ArgoCD, auto-syncing all 3-tier manifests on every Git commit, providing declarative deployments with a full audit trail and zero manual kubectl interventions in production.
- Built a full observability stack (Prometheus, Grafana, Loki, and Alertmanager) covering cluster metrics, pod logs, and application traces, with API latency dashboards and automated email alerting that reduced MTTD across all three tiers.
- Documented the complete system architecture using Mermaid diagrams covering the AWS infrastructure, Kubernetes cluster topology, CI/CD pipeline, GitOps workflow, and observability stack, making the project easy to hand off and maintain.
- Hardened cluster networking with restrictive AWS Security Groups, allowlisting only required ports (etcd 2379-2380, kubelet 10250, Canal VXLAN 8472 UDP) and enforcing least-privilege IAM and VPC-internal access throughout.

### Two-Tier Web Application Github | Live

*Next.js 15 • PostgreSQL 16 • Docker Compose • Jenkins • Nginx • AWS EC2 • Prisma • Certbot SSL*

- Engineered a 4-stage Jenkins CI/CD pipeline (build, test, deploy, verify) on AWS EC2 with consistent sub-2m-30s deployments and a fully automated release track.
- Implemented automated rollback using versioned Docker image tags and a last-known-good marker file, enabling health-check-triggered recovery with no manual intervention.
- Validated production resilience through failure drills: ~5s crash recovery (SIGTERM), ~20s DB outage with graceful 503 degradation, and ~116s full EC2 reboot recovery, all without manual intervention.
- Documented findings through RCA reports and operational runbooks covering all three failure scenarios.
- Secured Postgres credentials via Jenkins credential binding, keeping secrets out of source control across all deployment stages.

### MeroCV Live

*Next.js • PostgreSQL • Docker Compose • Turborepo • Bash • Python*

- Reduced production Docker image size by ~90% across a 2-app Next.js monorepo using multi-stage Dockerfiles with Turborepo pruning, copying only the standalone bundle into the runner stage.
- Eliminated race-condition failures across 2 interdependent services by configuring Docker Compose with healthcheck-gated startup ordering, isolated bridge networking, and environment-variable injection.
- Architected RBAC and a reusable component system for a production resume builder, maintaining deployment runbooks and SOPs that reduced contributor onboarding time.

---

## EDUCATION

### Bachelor of Science in Computer Science and Information Technology (BSc. CSIT)

*Bhaktapur Multiple Campus, Bhaktapur, Nepal*

**2022 - 2026**